



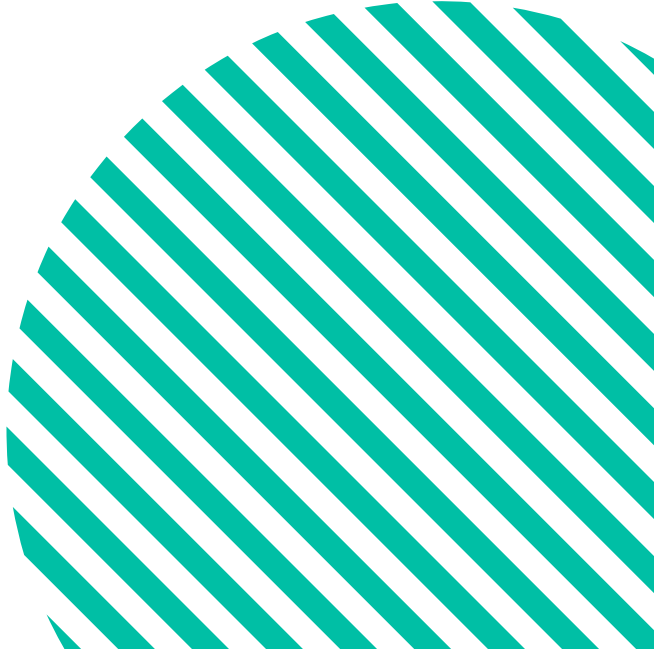
Université de Montpellier

HAI926I

Embedded Databases and Persistence

Presented by:

PHAM Minh Hang
Janna GUDUMAC





“When you close an app and open it again, how are your tasks still there?”

MiniTodo: Our Running Example

MiniTodo

New task

ADD

3 tasks · 1 completed

☐ Rehearse the demo  

☒ Read the course notes  

☐ Prepare the presentation  

C



Add a task

Type a title, press ADD → a new row in the database

R



Edit its title

Tap the pencil, change the title, tap SAVE

U



Mark it completed

Tick the checkbox → completed changes from 0 to 1

D



Delete it

Tap the bin and confirm → the row is removed



Typed text is not saved data. Each task = a title + completed or not. Typed text only becomes a task when you press ADD.

Why Does Data Need to Persist?



Process memory (RAM)

Like a whiteboard: wiped after class

- Holds the Java objects and variables of the app's running process
- Very fast — but temporary
- **Wiped when the process ends:** the user closes the app, Android kills it to free memory, a force-stop, or the phone shuts down
- Fine for temporary calculations only



Persistent storage

Like a notebook: still there tomorrow

- The phone's flash memory: files and database records
- Slower — but survives shutdown
- **A later run of the app** can read the data back, even days later
- The app rebuilds new objects from the stored data

Persistence = writing data to persistent storage so a later execution can read it back. The next run does not get the old objects back: it reads the stored data and builds new objects with the same values.

Persistence Mechanisms on Android

MECHANISM	USED FOR	EXAMPLE	IN MINITODO
 Saved UI state (Bundle)	Small, temporary screen state	Unfinished text during rotation	Text typed in taskInput
 SharedPreferences / DataStore	A few simple settings (key-value)	Dark mode on / off	Not used
 Files	Documents, images, exports	A tasks.json export, a photo	Not used
 Embedded database (SQLite + Room)	Many structured records, searched and changed often	A task list	All tasks, in tasks.db
 Remote server	Sharing data across devices	A cloud to-do app	Not used

Choose by asking: *what shape is the data, and how long must it live?*

Why not SharedPreferences? *Only key-value pairs — not records you sort, search and edit one by one.*

Activity and UI State Restoration

1

TYPE TEXT

The user types into taskInput but does not press ADD

2

ROTATE DEVICE

A configuration change: the screen size and orientation change

3

ACTIVITY IS RECREATED

Android destroys the old Activity and creates a new one

4

UI STATE IS RESTORED

The new Activity receives the saved Bundle and refills its views



What is an Activity?

- **One screen** of an app. MiniTodo has a single one: MainActivity.
- **Android controls its lifecycle:** it creates, pauses and destroys it.
- **Configuration changes** (rotation, language, dark mode) make Android recreate it.



Why does this matter?

The new orientation may need different layouts and resources, so Android redraws the screen from scratch. **Anything kept only in the old Activity's fields is gone** — unless it was saved somewhere first.

State Restoration in MiniTodo

activity_main.xml (real code, simplified)

```
<EditText
  android:id="@+id/taskInput"
  android:hint="@string/new_task"
  android:inputType="textCapSentences"
  android:maxLines="1" ... />
```

- **Why the ID matters:** remove android:id and Android doesn't know where to save the text — it is lost on rotation.
- **After a successful ADD,** the code calls `taskInput.setText("")` to clear the field.
- **Still only a draft:** after a force-stop or a fresh launch it may be gone. A saved task never is.

1 TYPE TEXT, DON'T PRESS ADD

2 ROTATE THE DEVICE

3 THE UNFINISHED TEXT IS RESTORED
Verified on the emulator — no custom code needed

4 NO ROW HAS BEEN INSERTED YET
Pressing ADD is a separate action: it creates a Task object and asks the database to insert a row

What survives which event?

EVENT	UNFINISHED TEXT (ADD NOT PRESSED)	SAVED TASK IN TASKS.DB
Rotate the device	Kept — EditText view state	Kept
Close and reopen the app	Not guaranteed	Kept
Force-stop (Settings › Apps)	No	Kept
Uninstall or clear storage	No	Lost — the file is deleted

A saved task survives everything except uninstalling the app or clearing its storage — once its write has completed.

Source: tested on an API 29 emulator (project README). Rebooting the phone keeps saved tasks too.

File Persistence on Android



APP-PRIVATE FILES

- Inside the app's own storage: `.../files/`
- Accessed with `getFilesDir()`
- For internal data
- Usually only this app reads them
- **Deleted** when the app is uninstalled



USER-VISIBLE DOCUMENTS

- Location chosen by the user
- Storage Access Framework: the system file picker
- For exports and documents
- Can be opened with other tools
- **Stay** after the app is uninstalled



TASKS.DB IS A FILE TOO

MiniTodo's SQLite database lives in the app's private storage:

`.../databases/tasks.db`

The difference: we never read or write its bytes ourselves — SQLite manages them for us.

FILES ARE USEFUL FOR

TEXT AND JSON

IMAGES, BINARY
CONTENT

EXPORTED
SNAPSHOTS

A file is just a sequence of bytes.

The format (text, JSON, PNG, PDF) gives the bytes their meaning.

Imagine storing tasks as JSON

```
[  
  { "id": 1,  
    "title": "Prepare the presentation",  
    "completed": false },  
  
  { "id": 2,  
    "title": "Read the course notes",  
    "completed": true }  
]
```



READABLE

Any tool or person can open and understand it



ALL 3 FIELDS

id, title and completed for each task — with real true / false values



GOOD FOR EXPORT AND SHARING

A portable snapshot of the whole list



BUT: A STATIC SNAPSHOT

Changing one task means rewriting the whole file

Now imagine: mark task number 2 as completed, in a list of 500 tasks...

When a file is not enough

SUPPOSE WE NEED TO

 Find by ID

 Update one task

 Delete one record

 Sort or filter

 Stay consistent

TASK 2 → COMPLETED	WITH A JSON FILE	WITH A DATABASE
Steps	Read the whole file → parse → find the task → change it → rewrite the whole file	UPDATE tasks SET completed = 1 WHERE id = 2;
With 500 tasks	All 500 records are read and written again	Only one row is touched
If the app dies mid-way	The file may be half-written and corrupted	The transaction rolls back — data intact
Who does the work	Our code: parse, search, rewrite	The database: queries, record-level operations

A database provides these operations directly. Files still have their place — but for structured records that change all the time, a relational database is the practical choice.

SQLite: An Embedded Relational



EMBEDDED

A library running inside the app's own process



SERVERLESS

No separate server, no network, nothing to install



RELATIONAL

Data in tables of rows and columns



LOCAL

The whole database is one file on the device



TRANSACTIONAL

Related changes commit all together, or not at all

CLIENT-SERVER (MySQL, PostgreSQL)



The app sends commands over the network to a server.

EMBEDDED (SQLite, MiniTodo)



No connection needed – that's why MiniTodo saves tasks offline.

SQLite is the storage engine. **Room** is the layer our Java code uses to access it.

The tasks Table

COLUMN	SQLITE TYPE AND CONSTRAINTS	MEANING
id	INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL	Unique per task, even if two titles are the same
title	TEXT NOT NULL	Task description; the app also rejects blank titles
completed	INTEGER NOT NULL	0 = false, 1 = true

EXAMPLE ROWS · one row = one task

ID	TITLE	COMPLETED
1	Prepare the presentation	0
2	Read the course notes	1
3	Rehearse the demo	0



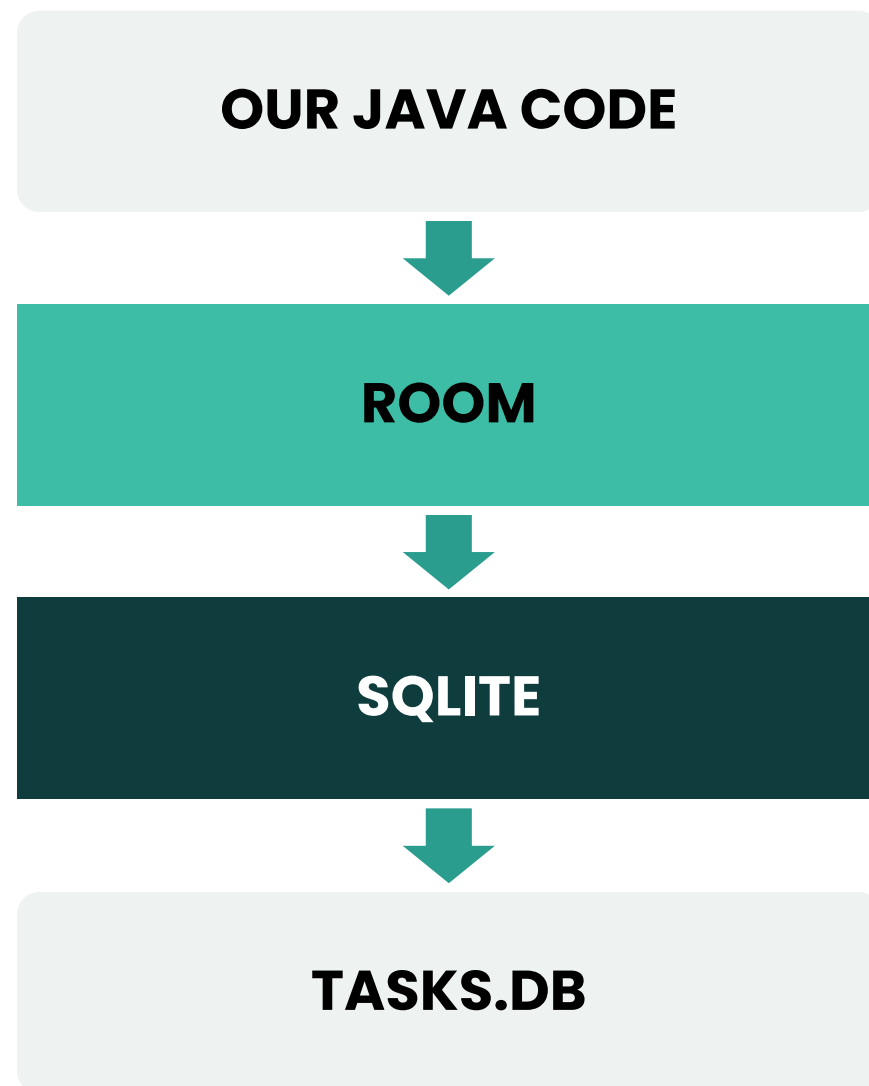
SQLITE HAS NO BOOLEAN

Java uses a boolean. Room stores false / true as 0 / 1 in an INTEGER column, and converts back when reading.

Room's generated code: entity.completed ? 1 : 0

Why not use the title as the key? Titles can repeat (two “Study” tasks) and can be edited. A key must be unique and stable.

Room: A Modern Abstraction over SQLite



Room is a Google Jetpack library. It does not replace SQLite — SQLite is still underneath.

SQLite — the engine	Room — the layer on top
Stores and queries records	Connects our Java code to SQLite
Executes SQL	Generates the database access code
Manages the file and transactions	Maps classes to tables, objects to rows
Built into Android	Checks @Query SQL when you build

Without Room

SQL written as strings, each column read from a Cursor, objects built by hand. A typo in a column name only crashes when the app runs.

With Room

Classes, interfaces and annotations. Room generates the code (TaskDao_Impl.java) and checks queries at build time. We still write SQL for our read query.

Room's Programming Model



@Entity

ENTITY TASK

What is stored?

Describes the tasks table and its columns. One Task object = one row.

Analogy: *The record form template*



@Dao

DAO TASKDAO

How is it accessed?

An interface declaring the 4 operations: @Insert, @Query, @Update, @Delete — with no method bodies.

Analogy: *The librarian who adds, finds, edits and removes records*



@Database

DATABASE APPDATABASE

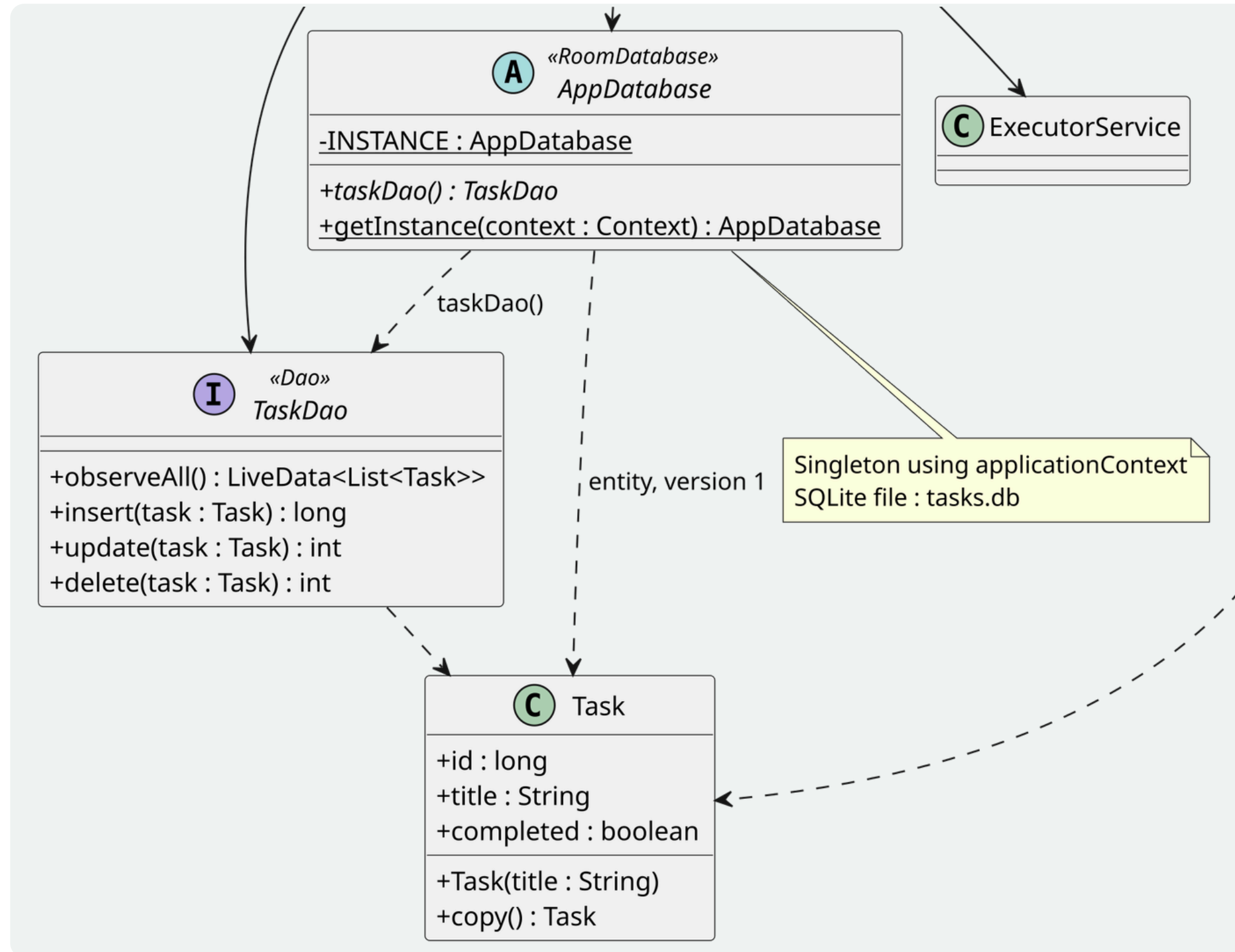
Where do we get access?

Lists the entities, schema version 1 and the file tasks.db. Provides the DAO. One instance only.

Analogy: *The library building where you meet the librarian*

At compile time Room reads these annotations, generates `TaskDao_Impl` and `AppDatabase_Impl`, and checks the SQL in @Query: a wrong table or column name fails the build.

Zoom: the three Room classes in the diagram



TASK — C

A plain class = the Entity. Public fields `id`, `title`, `completed`, plus `Task(title)` and `copy()`.

TASKDAO — I

An interface: the 4 CRUD methods. `insert` returns `long` (new id); `update` and `delete` return `int` (rows changed).

APPDATABASE — A

An abstract class. `taskDao()` in italics = abstract: Room writes the body.

UNDERLINED = STATIC

`INSTANCE` and `getInstance()`: a singleton — one database, file `tasks.db`, for the whole app.

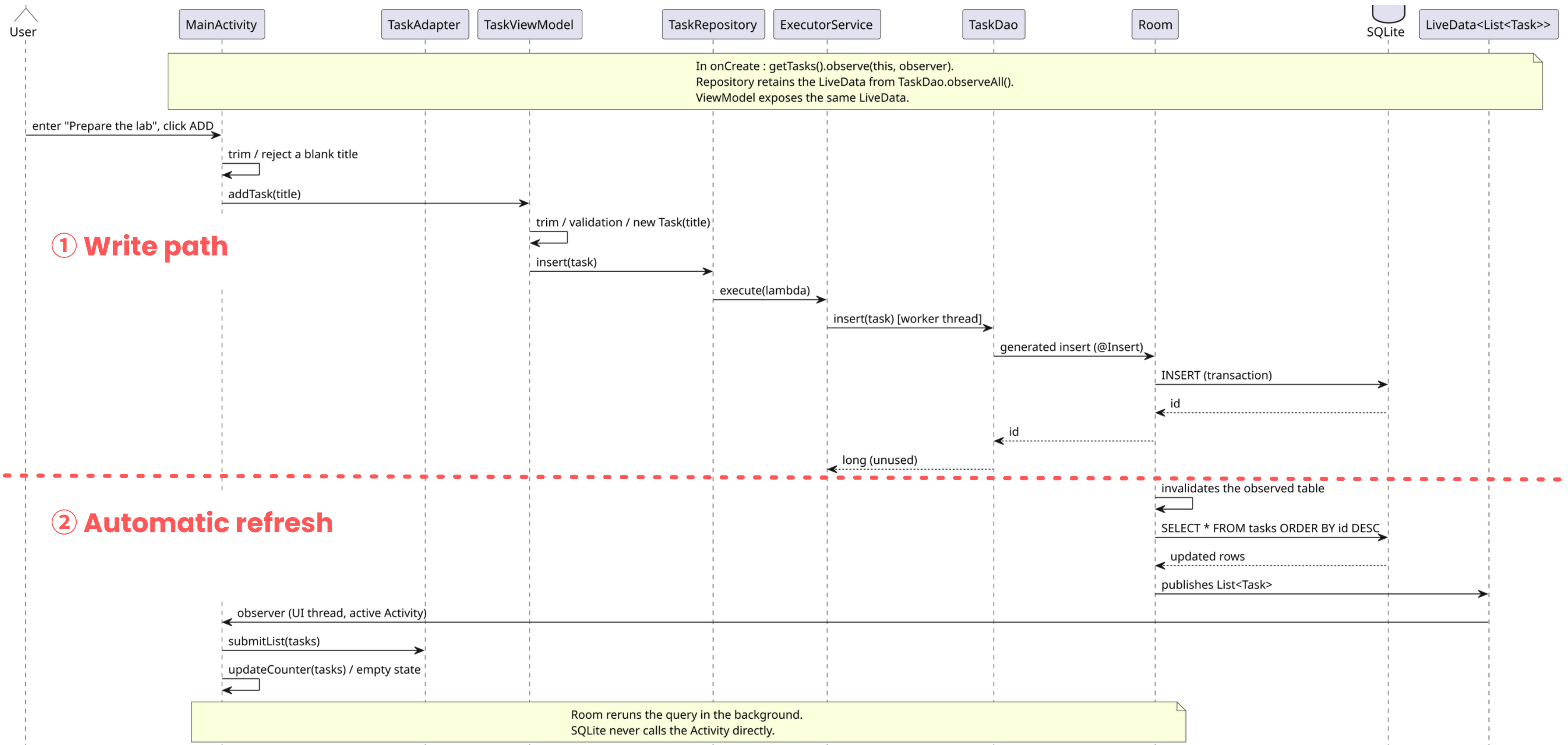
From User Actions to Database Operations

USER ACTION	DB OPERATION	DAO METHOD	SQL ROOM RUNS
C PRESS ADD	Insert a row	@Insert insert()	INSERT INTO tasks ...
R DISPLAY THE LIST	Query rows	@Query observeAll()	SELECT * FROM tasks ORDER BY id DESC
U CHECK / EDIT THE TITLE	Update a row	@Update update()	UPDATE tasks SET ... WHERE id = ?
D DELETE (AFTER CONFIRMING)	Delete a row	@Delete delete()	DELETE FROM tasks WHERE id = ?

- **Check and edit both call update():** the app copies the task with the same id (task.copy()), changes one field, and Room overwrites the row found by its primary key.
- **Cancelling a dialog writes nothing** — no arrow ever reaches the database.

CRUD = Create, Read, Update, Delete — the foundation of almost every application.

UML sequence diagram: adding a task



From a Java Object to a Relational Row

Task.java (relevant fields only; constructor and copy() omitted)

```
@Entity(tableName = "tasks")
public class Task {
    @PrimaryKey(autoGenerate = true)
    public long id;
    @NonNull
    public String title;
    public boolean completed;
}
```

- **@Entity** names the table
- **@PrimaryKey(autoGenerate)** = auto-generated key
- **@NonNull** becomes NOT NULL

Same logical record, two forms — this is ORM (Object-Relational Mapping).

JAVA FIELD	SQLITE COLUMN
long id	INTEGER id PRIMARY KEY AUTOINCREMENT
String title + @NonNull	TEXT title NOT NULL
boolean completed	INTEGER completed (0/1)

Write: field values → columns

Read: query rows → new Task objects

Rebuilding objects from rows is how tasks come back each time the app opens.

TaskDao: Declarative Database Access

```
@Dao
public interface TaskDao {
    @Query("SELECT * FROM tasks ORDER BY id DESC")
    LiveData<List<Task>> observeAll();

    @Insert
    long insert(Task task);

    @Update
    int update(Task task);

    @Delete
    int delete(Task task);
}
```

Create → @Insert

Read → @Query

Update → @Update

Delete → @Delete

- We declare operations using annotations.
- Room generates the implementation during compilation.
- Queries use SQL; writes operate on Task objects.

AppDatabase:

Creating and Reusing the Database

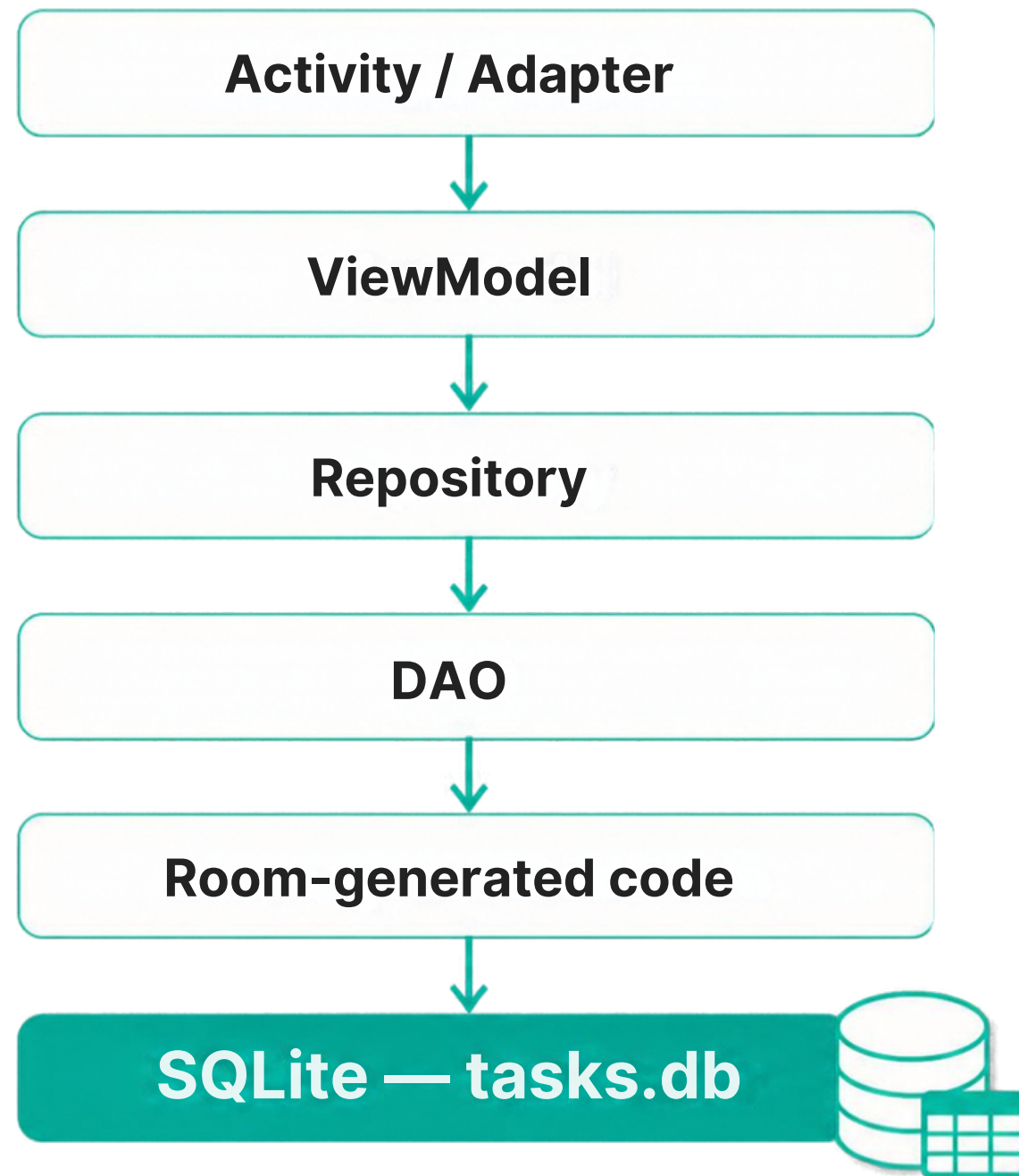
```
@Database(entities = {Task.class}, version = 1, exportSchema = true)
public abstract class AppDatabase extends RoomDatabase {
    private static volatile AppDatabase INSTANCE;

    public abstract TaskDao taskDao();

    public static AppDatabase getInstance(Context context) {
        if (INSTANCE == null) {
            synchronized (AppDatabase.class) {
                if (INSTANCE == null) {
                    INSTANCE = Room.databaseBuilder(context.getApplicationContext(),
                        AppDatabase.class, "tasks.db").build();
                }
            }
        }
        return INSTANCE;
    }
}
```

- **RoomDatabase:** database entry point.
- **Singleton:** one shared instance per process.
- **taskDao():** access to task operations.
- **tasks.db:** SQLite database stored privately by the app.

End-to-End Architecture



The **Activity** handles user interaction.

The **ViewModel** validates task titles.

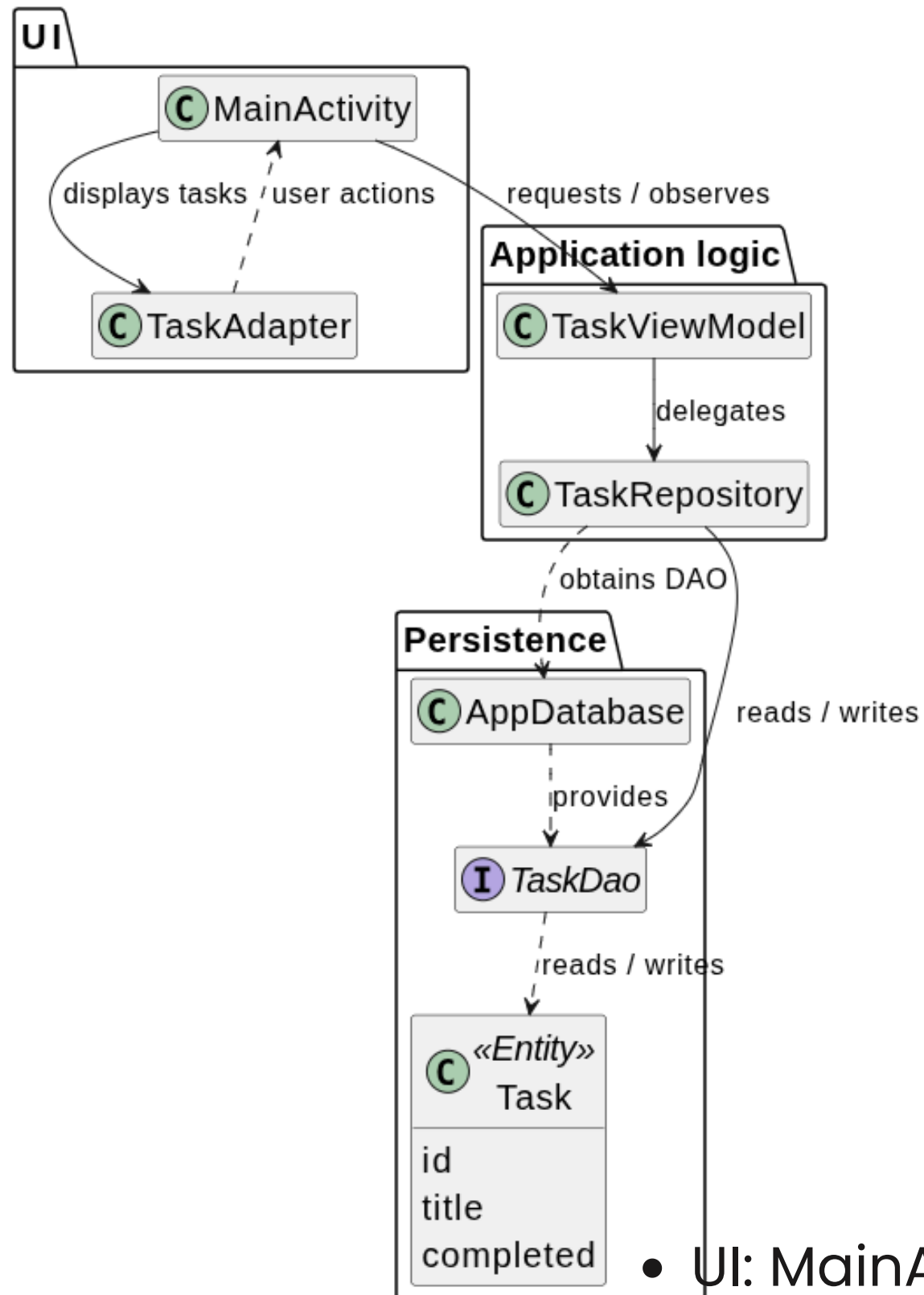
The **Repository** manages data access and schedules writes.

The **DAO** declares the database operation. Room's generated implementation executes it against SQLite.

For reading, the Repository retains the DAO's LiveData, and the ViewModel exposes that same object to the Activity.

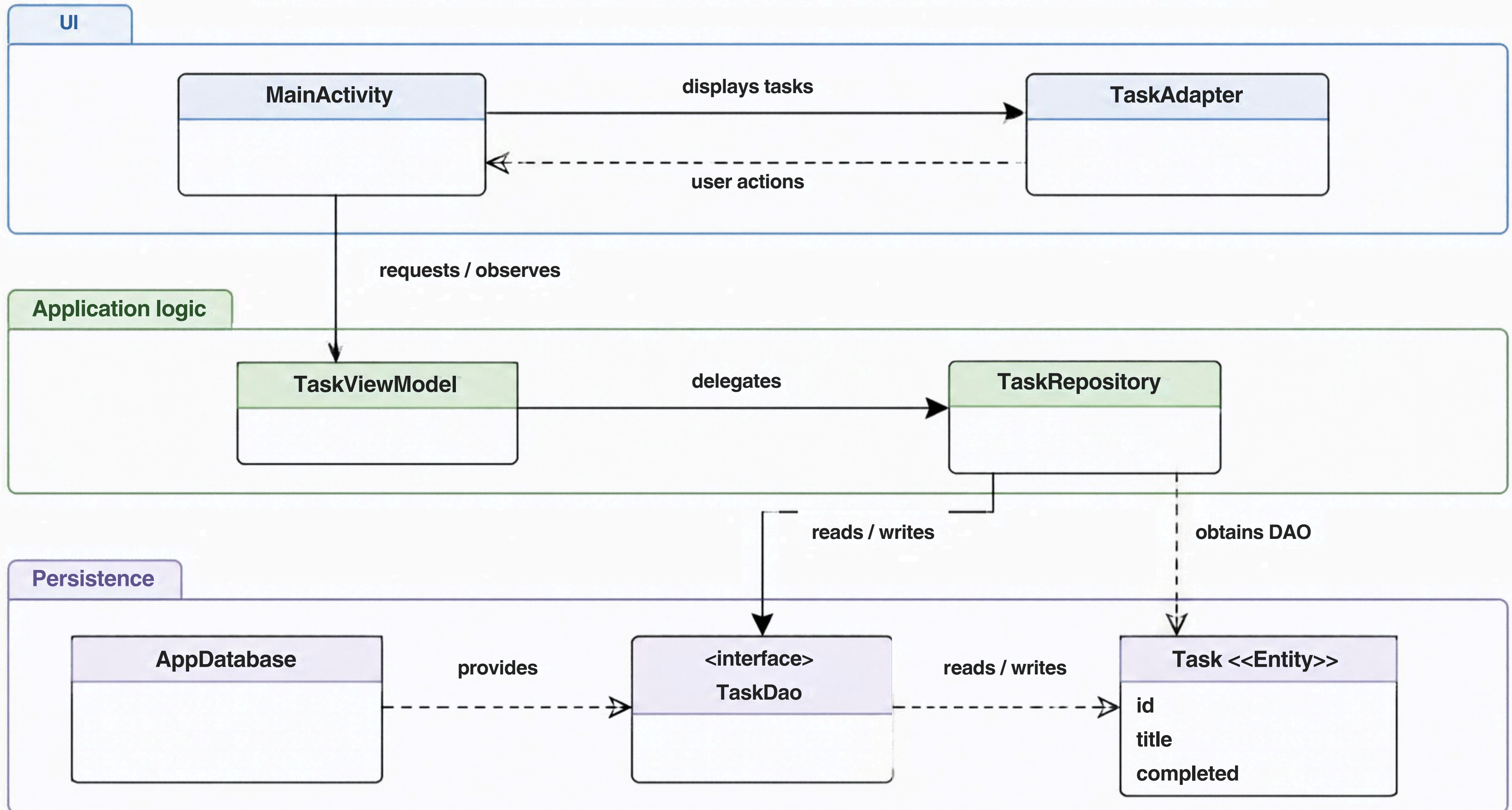
UML Class Diagram

MiniTodo — responsibilities and dependencies



- UI: MainActivity, TaskAdapter, Listener
- Coordination: TaskViewModel, TaskRepository
- Persistence: Task, TaskDao, AppDatabase

Unit 3: Design Patterns: MVC and MVVM





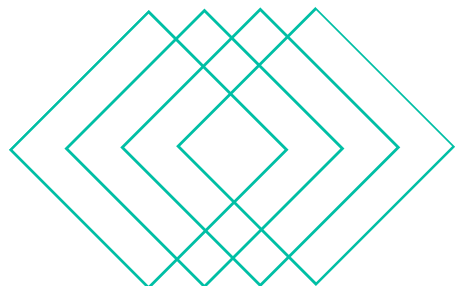
Responsibilities and Dependencies



Component	Responsibility
Activity	Input, dialogs, observing data
Adapter	Display rows and report interactions
ViewModel	Validate titles and expose task data
Repository	Coordinate access and background writes
DAO	Declare queries and write operations

Why separate them?

- Each component has a focused responsibility.
- Changes remain easier to locate.
- UI components do not need SQL or thread-management details.





Adding a Task — UML Sequence Diagram

- 1.ADD click and title validation.
- 2.Task creation and queued write.
- 3.SQLite insertion.
- 4.Observed query refresh.
- 5.New list displayed.



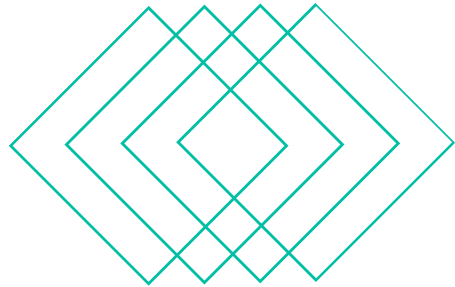


One Pipeline, Multiple Operations



User action	DAO operation
Add task	@Insert
Edit title	@Update
Check/uncheck	@Update
Delete task	@Delete

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





The Repository as a Persistence Boundary

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





Threading Model: UI Thread vs Worker

Thread

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





Updating and Deleting — Same Architecture

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





Reactive Reads: Room + LiveData

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





What Persists Where in MiniTodo?

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





Demo 1 — State Restoration and JSON

Export

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





Demo 2 — CRUD with Database Inspector

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





Demo 3 — Does the Data Really Survive?

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





Your Turn: Questions

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





Key Takeaways

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





BACKUP SLIDES ->

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





BACKUP SLIDES ->

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





BACKUP SLIDES ->

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





SQLiteOpenHelper vs Room

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





Cursor vs Room DAO

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





Schema evolution / migrations

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.





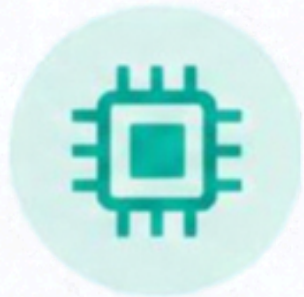
Files vs relational databases

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque non elit mauris. Cras euismod, metus ac finibus finibus, felis dui suscipit purus, a maximus leo ligula at dolor.



What Is Persistence?

Persistence means keeping data after the app closes or the device restarts.



Memory / RAM

- Fast
- Temporary
- Lost on process death



Durable storage

- Survives restarts
- Files or database
- Available next launch



Persistence turns temporary app state into lasting data.

Where Android Data Can Live



Room

For structured relational data and queries.



DataStore

For small preferences and settings.



Files

For documents, exports, and media.



UI state

For temporary screen state. Not intended for durable persistence.



The right solution depends on the data structure, size, and required lifetime.

Why Room for MiniTodo?

MiniTodo stores tasks locally and needs them to remain available after the application is closed. Room provides a structured persistence layer on top of SQLite.



Structured Task Records

MiniTodo stores tasks with a defined structure: an identifier, a title, and a completion state.



Reliable CRUD Operations

Room makes it easy to create, read, update, and delete tasks through a DAO.



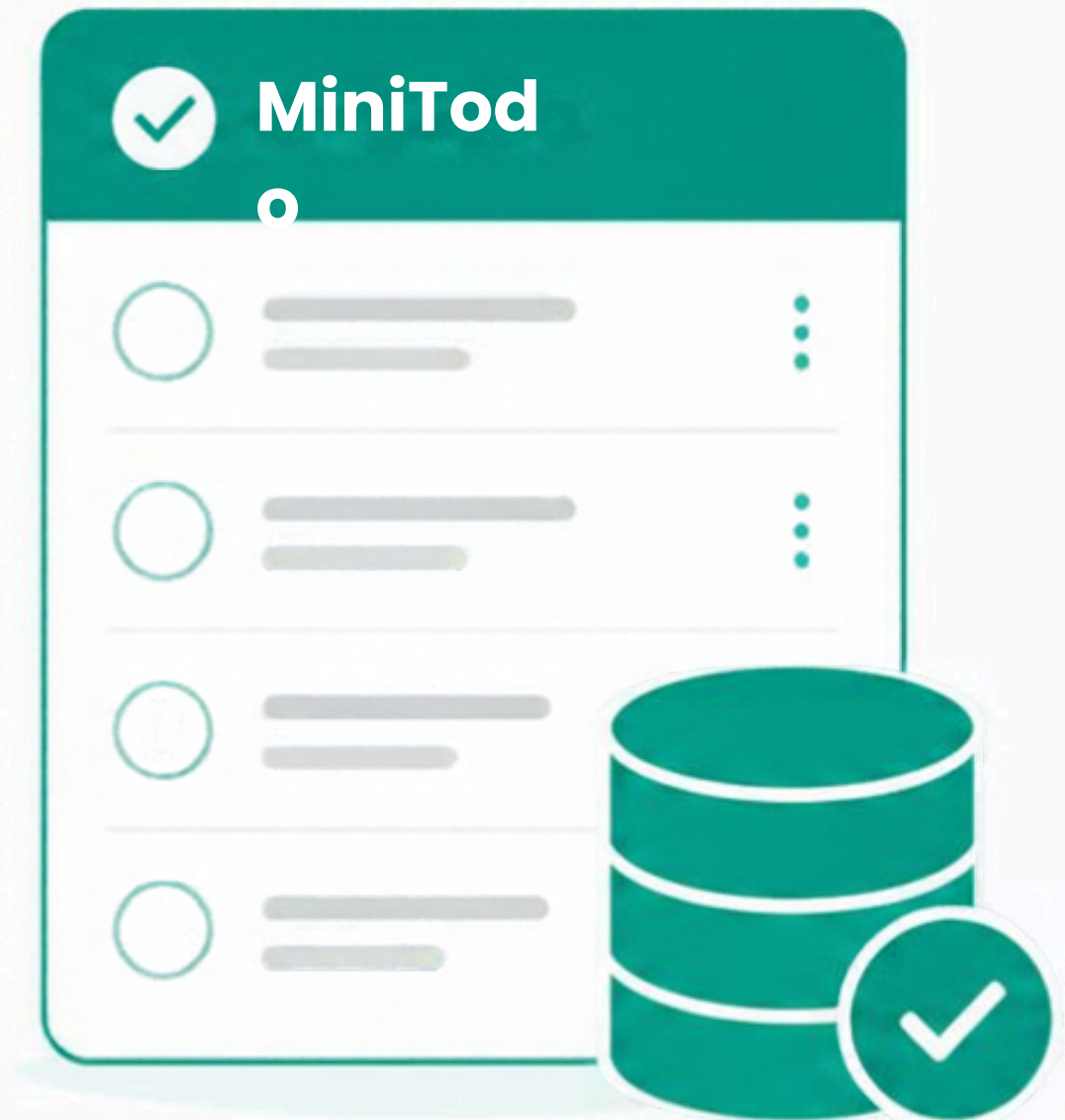
Survives App Restarts

Tasks are stored locally and remain available when the application is closed and reopened.



Safer Database Access

Room maps Java/Kotlin objects to database tables and checks SQL queries at compile time.



**Room provides a safer and more structured API over
SQLite.**

SQLite: An Embedded Database



SQLite is a lightweight relational database engine embedded inside the app. It requires no separate server.

Data is organized in **tables**, with **rows** and **columns**. You use **SQL queries** to create, read, update, and delete data.

Everything is **stored in the app's database file**.



The database runs locally on the device.

The tasks Table

Task	Completed
Task 1	0
Task 2	1
Task 3	0
Task 4	1
Task 5	0
Task 6	1
Task 7	0
Task 8	1
Task 9	0
Task 10	1

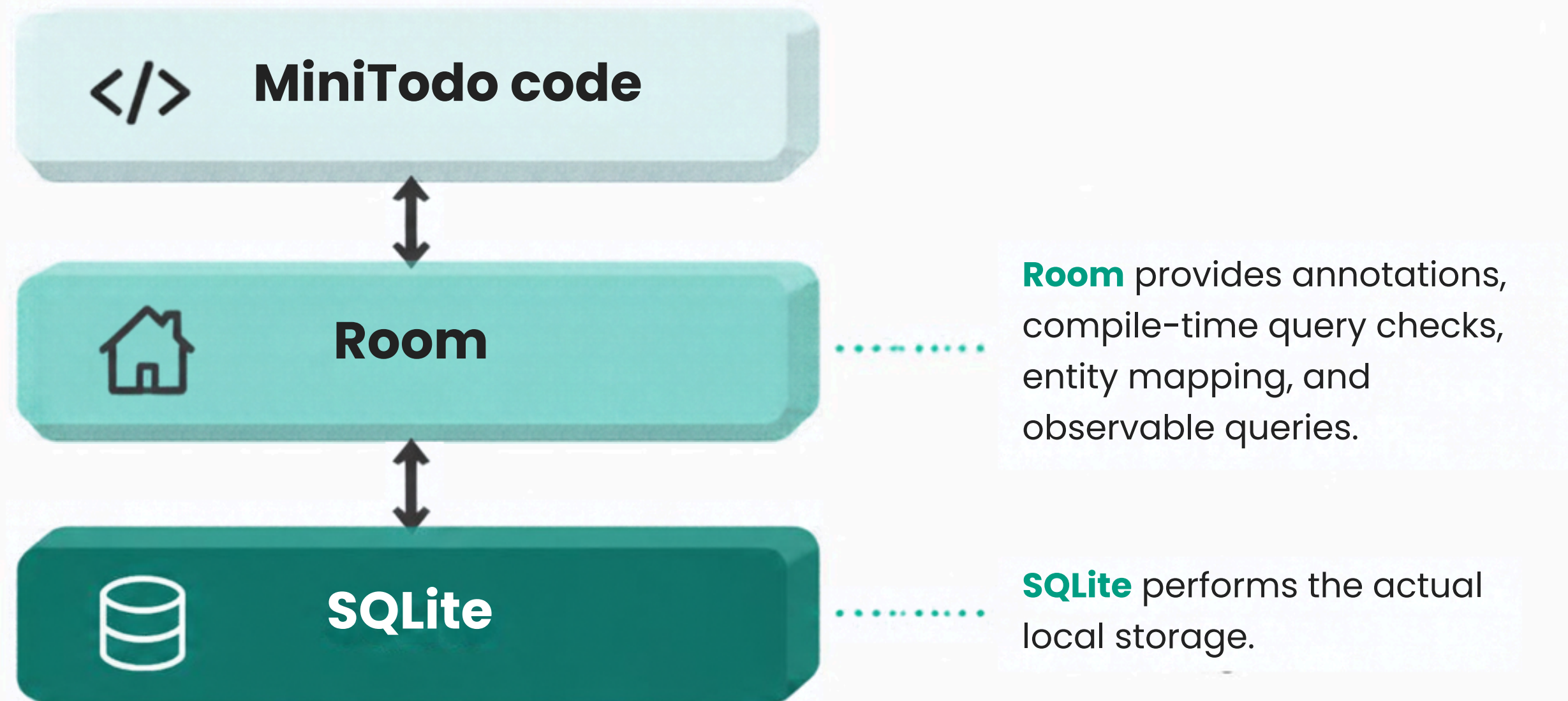
0

incomplete

1

complete

Room: A Layer on Top of SQLite



Takeaway

Room is not a replacement for SQLite; it is an abstraction layer over it.

